VLSI Design Internship – Task 2

Introduction to Verilog HDL and RTL Design of Basic Combinational Circuits

Part A: Introduction to Verilog HDL

Objective

In Task-2, the first practical step into VLSI implementation was taken by learning Verilog Hardware Description Language (HDL), Register Transfer Level (RTL) design concepts, and by writing, simulating, and verifying combinational circuits using Verilog. This marks the transition from schematic-level thinking to code-based chip design.

Circuits Implemented

- Logic Gates (AND, OR, NOT, NAND, NOR, XOR, XNOR)
- Half Adder
- Full Adder

Tool Used

ModelSim SE-64 10.6d – used to write, compile, and simulate Verilog RTL code, and to view waveform outputs.

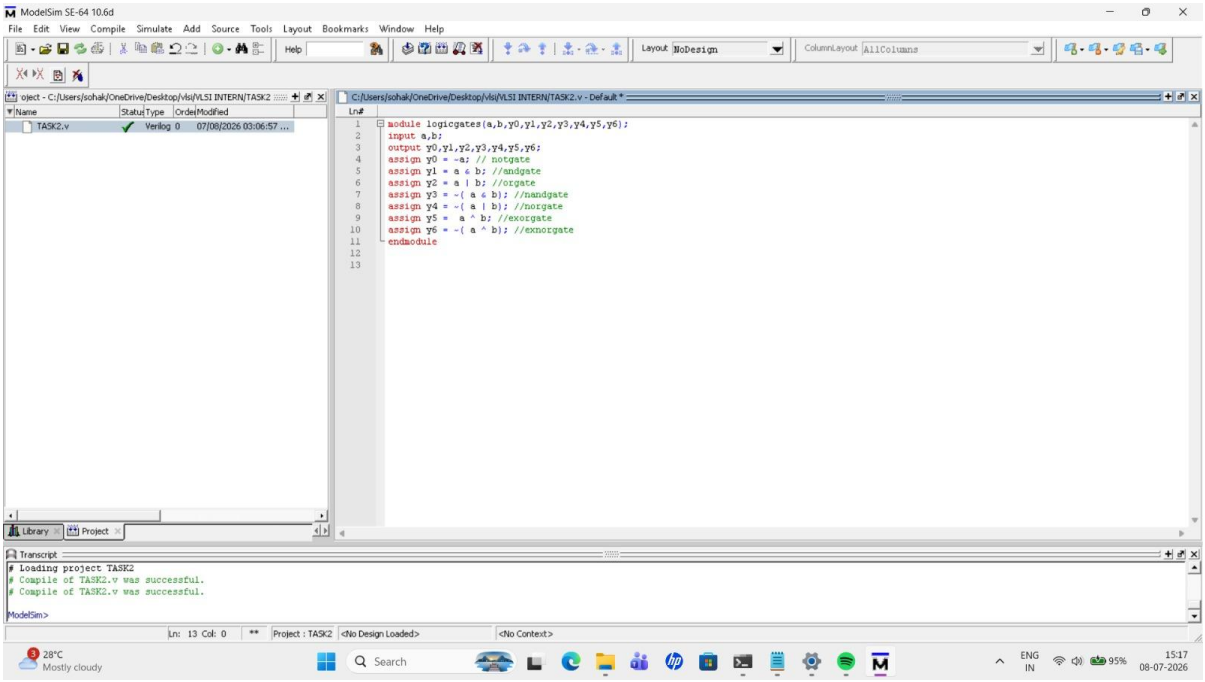
Part B: Logic Gates in Verilog

Verilog Code

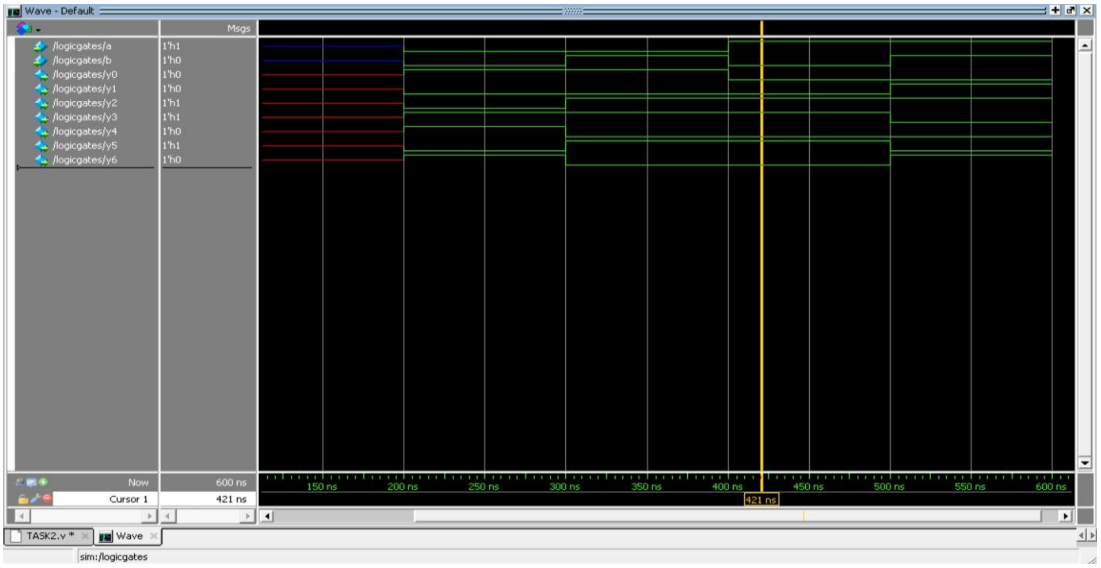
A single module logicgates(a,b,y0,y1,y2,y3,y4,y5,y6) was written to implement all basic logic gates using continuous assign statements:

- y0 = ~a (NOT) y1 = a & b (AND) y2 = a | b (OR)
- y3 = ~(a & b) (NAND) y4 = ~(a | b) (NOR)
- y5 = a ^ b (XOR) y6 = ~(a ^ b) (XNOR)

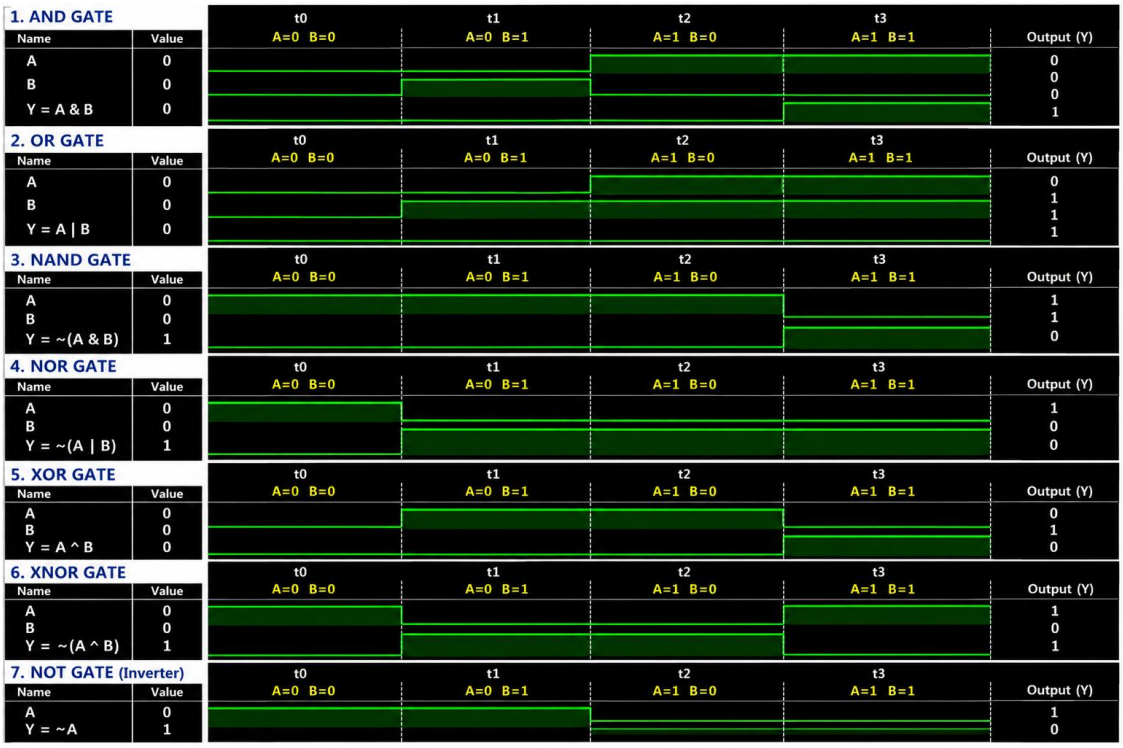
Screenshot 1: Logic Gates Verilog Code (ModelSim)



Screenshot 2: Logic Gates Simulation Waveform



Screenshot 3: Logic Gates – Truth Tables and Waveform Reference



LOGIC TRUTH TABLES

AND GATE: Y = A & B

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR GATE: Y = A | B

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

NOT GATE: Y = ~A

A	Y
0	1
1	0

NAND GATE: Y = ~(A & B)

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

NOR GATE: Y = ~(A | B)

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

XOR GATE: Y = A ^ B

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

XNOR GATE: Y = ~(A ^ B)

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

Testbench for Logic Gates

A testbench was written to apply all four input combinations of a and b to the logicgates module and observe every gate output:

```
module tb_logicgates;

    reg a, b;
    wire y0, y1, y2, y3, y4, y5, y6;

    logicgates uut (
        .a(a), .b(b),
        .y0(y0), .y1(y1), .y2(y2), .y3(y3),
        .y4(y4), .y5(y5), .y6(y6)
    );

    initial begin
        a = 0; b = 0; #10;
        a = 0; b = 1; #10;
        a = 1; b = 0; #10;
        a = 1; b = 1; #10;
        $finish;
    end

endmodule
```

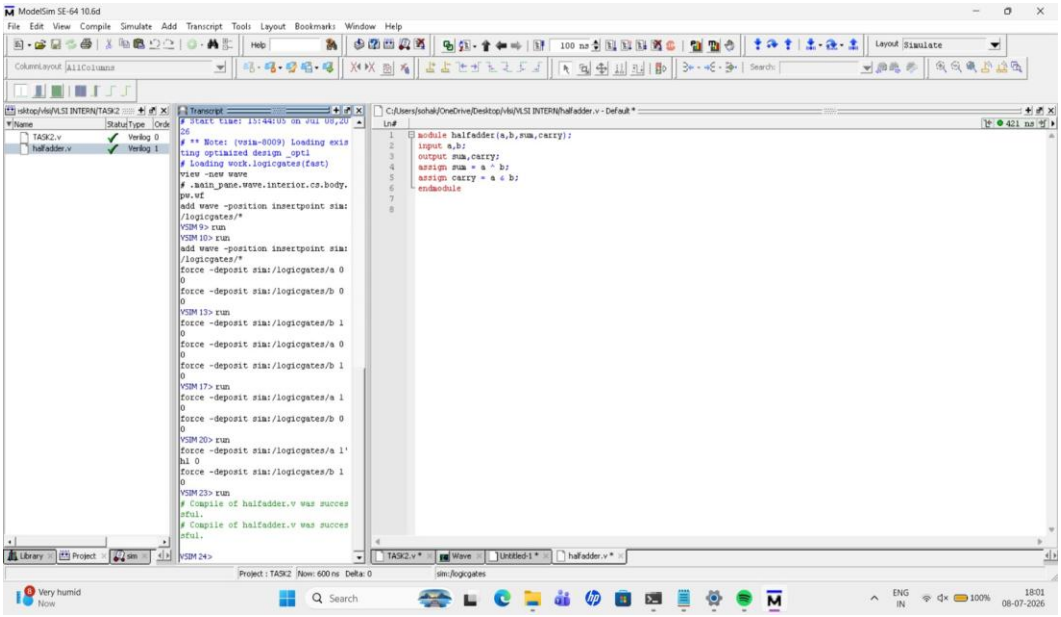
Part C: Half Adder (RTL Design)

Verilog Code

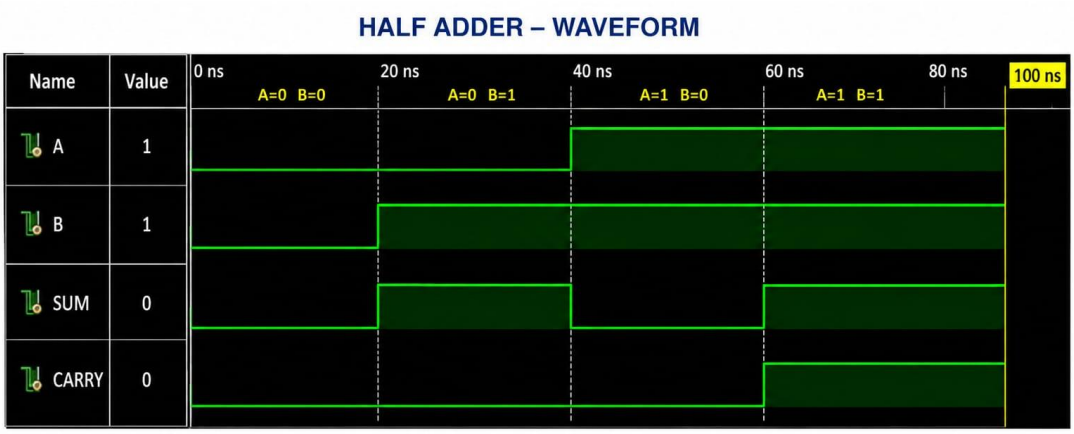
Module halfadder(a, b, sum, carry) was written as:

- sum = a ^ b carry = a & b

Screenshot 4: Half Adder Verilog Code and Simulation Log



Screenshot 5: Half Adder Waveform and Truth Table



HALF ADDER TRUTH TABLE

A	B	SUM (A XOR B)	CARRY (A AND B)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

HALF ADDER TRUTH TABLE

A	B	SUM	CARRY
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Testbench for Half Adder

A testbench was written to apply all four input combinations of a and b to the halfadder module and observe sum and carry:

```
module tb_halfadder;

    reg a, b;
    wire sum, carry;

    halfadder uut (
        .a(a),
        .b(b),
        .sum(sum),
        .carry(carry)
    );

    initial begin
        a = 0; b = 0; #10;
        a = 0; b = 1; #10;
        a = 1; b = 0; #10;
        a = 1; b = 1; #10;
        $finish;
    end

endmodule
```

Part D: Full Adder (RTL Design)

Verilog Code

Module full_adder(A, B, Cin, Sum, Cout) was written as:

- Sum = A ^ B ^ Cin
- Cout = (A & B) | (B & Cin) | (A & Cin)

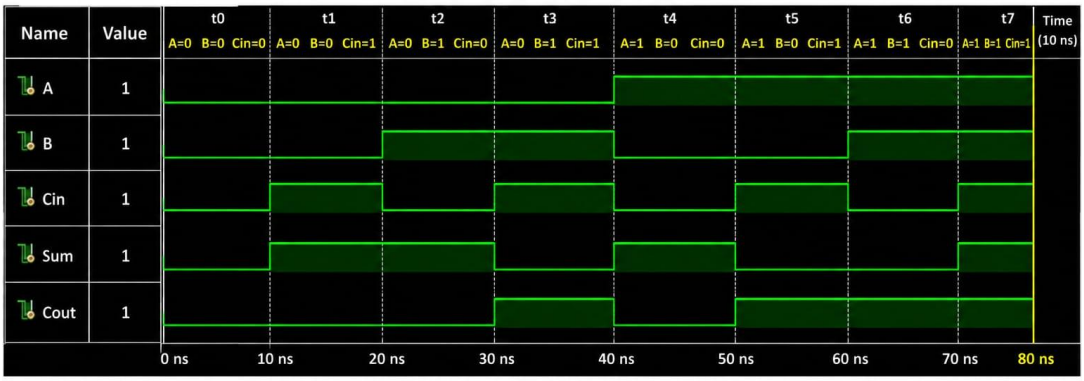
Screenshot 6: Full Adder Verilog Code

C:/Users/sohak/OneDrive/Desktop/vlsi/VLSI INTERN/work/fulladder1.v - Default *

```
Ln#
1   module full_adder(
2       input A,
3       input B,
4       input Cin,
5       output Sum,
6       output Cout
7   );
8
9   assign Sum = A ^ B ^ Cin;
10  assign Cout = (A & B) | (B & Cin) | (A & Cin);
11
12  endmodule
```

Screenshot 7: Full Adder Waveform and Truth Table

FULL ADDER – WAVEFORM



FULL ADDER TRUTH TABLE

A	B	Cin	Sum (A ⊕ B ⊕ Cin)	Cout ((A & B) | (B & Cin) | (A & Cin))
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

FULL ADDER TRUTH TABLE

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Testbench for Full Adder

A testbench was written to apply all eight input combinations of A, B and Cin to the full_adder module and observe Sum and Cout:

```
module tb_full_adder;

    reg A, B, Cin;
    wire Sum, Cout;

    full_adder uut (
        .A(A),
        .B(B),
        .Cin(Cin),
        .Sum(Sum),
        .Cout(Cout)
    );

    initial begin
        A = 0; B = 0; Cin = 0; #10;
        A = 0; B = 0; Cin = 1; #10;
        A = 0; B = 1; Cin = 0; #10;
        A = 0; B = 1; Cin = 1; #10;
        A = 1; B = 0; Cin = 0; #10;
        A = 1; B = 0; Cin = 1; #10;
        A = 1; B = 1; Cin = 0; #10;
        A = 1; B = 1; Cin = 1; #10;
        $finish;
    end

endmodule
```

Part E: Simulation and Verification

All modules were compiled and simulated in ModelSim SE-64 10.6d. Inputs a and b (and Cin for the full adder) were forced to all possible logic combinations, and the resulting waveforms were compared against the expected truth tables. The testbenches above were used to automate this process by applying every input combination with a #10 delay between each.

What Was Verified

- Outputs matched the expected truth table for every input combination
- No unknown (X) or floating states were observed
- Correct timing and transition of signals on the waveform
- Testbenches applied all possible input combinations to each module

Part F: Learning Outcomes

- Understood the basics of Verilog HDL and RTL design concepts
- Learned the structure of a Verilog module and continuous assignment statements
- Implemented all basic logic gates (AND, OR, NOT, NAND, NOR, XOR, XNOR) using Verilog
- Designed and verified a Half Adder and a Full Adder at RTL level
- Wrote testbenches to automatically verify each module against its truth table
- Simulated designs and analyzed waveform outputs using ModelSim
- Verified functional correctness of each circuit against its truth table

Author
Soha Kalekhan
VLSI Internship Task 2